ChatGPT

Architecture–Manual Alignment Review for PlanO

Executive summary

Based on the **user-supplied audit plus the follow-on "code verification" notes you pasted**, the overall architecture and the operations manual/blueprint are **directionally correct**: the system is a solid **geometry + selection** foundation that cannot yet operate as a secure revenue platform because **pricing, payments, authentication, tenancy, and durable persistence are missing**.

Two important alignment updates emerge from your verification:

- A key "gap" flagged earlier (service-gating to painting only) is **already resolved in code**: `handleCatalogServiceSelected` now supports **all four area services**, and `isPaintSelector` has been renamed to `isAreaServiceSelector`. That means the manual should **remove that task** and reallocate time to higher-risk work (security and revenue plumbing).
- The **postMessage security risk is worse than "just replace `*`"**: your verification shows an **engine-side receiver that does not validate** `event.origin` (`renderer.tsx:117`), which is explicitly warned against by Mozilla [1]'s MDN guidance: always verify sender identity via `origin` (and possibly `source`) and avoid `targetOrigin="*"`. [2]

Bottom line: **the architecture is not misinterpreted**, but the manual should be **amended** in a few concrete places before you "finalize and start," primarily around **postMessage origin strategy**, **cookie/JWT + CSRF posture**, and **how strict you are about "Phase 1 = revenue" vs "Phase 1 = internal e2e test."**

Architecture alignment

The current architecture described by your audit remains a strong fit for the intended platform:

- **Single iframe geometry engine** with business UI in PlanO UI is consistent with separation-of-concerns and your integration-lock premise (UI owns service/pricing; engine owns geometry). Your follow-on verification does not challenge this.
- The pipeline is accurately reflected as:
- Upload/raster/SceneJSON import: working
- 2D/3D edit + selection + ledger: working
- Pricing/quote/payment/confirmation: missing

Where the earlier manual needs correction is limited and specific:

- **Service gating**: the earlier blueprint/manual assumed gating was still painting-only; your verification says it's already widened to all 4 services, and selector logic was renamed accordingly. This is not a structural misinterpretation—just a stale assumption that should be removed from the roadmap.

What is still “architecturally true” and must not be violated

Even after the new code verification, the architecture must preserve these boundaries:

- The iframe engine should not become a business logic host.
- Any “money” or “identity” decisions must be server-authoritative, never browser-authoritative.
- The postMessage channel is a security boundary and must be treated like an API surface (origin allowlisting + schema validation). MDN explicitly warns that failure to check `origin` / `source` enables cross-site attacks, and that `targetOrigin="*"` can disclose data. [3]

Security invariants alignment

Your verification explicitly restates the “7 non-negotiable invariants” and concludes **0/7 are currently met**, which is consistent with the earlier manual’s warning that this is not production-ready as a financial system.

The two most urgent issues are:

postMessage origin controls

Your verification provides exact locations:

- `renderer.tsx:67` sends with `"*"`
- `renderer.tsx:117` receives without origin check
- `viewer3d.tsx` sends with `"*"` in multiple places
- `catalog-item.tsx` sends with `"*"`
- `PlannerFrame.tsx` sends with `"*"` but **does** validate `event.origin` inbound

This requires a two-sided fix:

1. **Outbound**: replace `"*"` with a specific `targetOrigin`. MDN: “Always provide a specific `targetOrigin`, not `*` ... Failing to provide a specific target could disclose data to a malicious site.” [4]
2. **Inbound**: validate `event.origin` (and ideally `event.source`) for every listener. MDN: always verify sender identity using `origin` and possibly `source`, and validate message syntax. [5]

Amendment needed (important): your proposed pattern “read `parentOrigin` from query params (or default localhost)” is *convenient* but can be **dangerous if used without an allowlist**, because it allows the engine to “trust” an origin that could be attacker-controlled if the engine URL is loaded outside your app. MDN’s guidance doesn’t forbid query-param configuration, but it requires *verifying sender identity*—so you should implement **an allowlist** derived from environment/config, not arbitrary URL input. [6]

Secrets exposure

Your verification indicates the RasterScan key remains in `01_raster/.env` and was committed historically. This must be handled as an exposure event.

OWASP [7]'s Secrets Management guidance explicitly notes that rotated/revoked secrets must be removed from exposed systems and warns that rewriting git history can have consequences. [8]
Stripe [9] similarly treats secret keys as credentials that must not be placed in source code, and recommends auditing repos for key patterns and planning rotation as a practiced process. [10]

Amendment needed: add "pre-commit secret scanning" and a formal rotation runbook to the manual as a mandatory pre-production gate (even if you don't adopt a full vault yet). [11]

File upload hardening

Your verification focuses on `01_raster/server/app.py` and identifies required controls (size limits, allowlist, auth gating). This aligns directly with OWASP's File Upload guidance: allowlisted extensions, file type validation, generated filenames, size limits, authorized-only uploads, store outside webroot, and CSRF protections. [12]

Amendment needed: once you move auth to cookies, the upload endpoint must be protected from CSRF as well (OWASP explicitly calls that out for upload surfaces). [13]

Authentication and tenant model alignment

Your verification proposes: JWT in **HttpOnly Secure SameSite cookies**, with `/api/auth/*` endpoints, Argon2id password hashing, and tenant scoping by `client_user_id = current_user.id`.

This is broadly aligned with the earlier manual, but it requires one critical correction:

Cookie-auth implies CSRF strategy

If authentication is cookie-based, state-changing endpoints are exposed to CSRF risks unless mitigated. The OWASP CSRF Prevention Cheat Sheet recommends robust CSRF token mechanisms and describes the Signed Double-Submit Cookie approach as recommended, emphasizing binding to session-specific data. [14]

So the manual must explicitly add:

- A CSRF token strategy for cookie-auth endpoints (projects save, quote lock, create checkout session, upload plan). [15]
- Prefer CSRF token in a custom header set by your frontend for same-origin requests. [15]

Session cookie flags

Your verification lists `HttpOnly`, `Secure`, `SameSite=Lax`, which is consistent with OWASP session guidance: - `Secure` is mandatory to prevent session ID disclosure over plaintext/mixed contexts. [16] - `SameSite` mitigates cross-origin leakage and provides some CSRF protection. [17]

Amendment needed: explicitly define how you handle local dev (HTTP) vs prod (HTTPS); in production, `Secure` is non-negotiable. [16]

Tenant isolation

Your scoping plan ("never `WHERE id = :id` alone; always filter by authenticated owner") precisely matches OWASP's multi-tenant guidance: do not trust client-supplied tenant IDs; bind tenant context to authenticated session; prevent IDOR by tenant-scoped querying; and use non-guessable identifiers. [18]

Amendment needed: include a "tenant context injection" test suite in the manual, because OWASP lists tenant context injection as a key multi-tenant risk. [18]

Pricing and payments alignment

Your verification confirms the earlier manual's core point: **pricing and payments are missing entirely** (`03_invoice/` and `04_stripe/` empty; `Estimate.tsx` hardcodes rate=0). That means the product is not yet revenue-operational.

The proposed quote lifecycle (Draft → Calculated → Locked → Paid) is correct for a payments system because it enforces server authority and prevents UI tampering, and it maps to Stripe best practices:

- Stripe warns that **webhooks are required for fulfillment**; you cannot rely on the success page being visited. [19]
- Stripe requires signature verification using the raw body, `Stripe-Signature` header, and endpoint secret. [20]
- Stripe supports idempotency keys for safely retrying POST requests. [21]
- Stripe Checkout Sessions are created on your server and you redirect using the session URL. [22]

Amendment needed (important): your "create-checkout-session" pseudocode should explicitly enforce that: - quote is `status="locked"` - totals come from DB (not client) - idempotency key ties to `quote_id` (or a derived stable token) - webhook handler is idempotent using a unique constraint on `(provider,event_id)` or `(provider,event_id)` in `webhook_events`. [23]

What must be amended before finalizing and starting work

This section answers your actual question: "Does the architecture and manual match, or is something misinterpreted and must be amended?"

Confirmed matches

The manual and current architecture match on the key truths:

- The geometry/selection pipeline is already strong and should remain isolated in the iframe engine.
- The platform is blocked at pricing/payments/auth/persistence, consistent with both the audit and your verification.
- The intended backend expansion via the existing FastAPI service is a sensible consolidation path (especially for an MVP).

Corrections and amendments

A concise “delta table” of what should change in the manual:

Area	Earlier manual assumption	Verified reality	Amendment
Service gating	Likely painting-only gating	Gate already widened; selector renamed	Remove task; keep regression test only
postMessage	“Replace `*` with strict origin”	Engine receiver lacks `event.origin` check; multiple `*` senders remain	Add receiver-side origin validation in engine + allowlist strategy; validate schema; do not accept query-param origin unless allowlisted [6]
Auth approach	“Auth could be deferred for MVP”	You’re positioning this as a financial system	Move DB+auth into Phase 1, not Phase 3; otherwise you can’t enforce tenant scoping or protect uploads
Cookie JWT	Mentioned but not fully specified	Plan is JWT in HttpOnly cookie	Add explicit CSRF defense plan for state-changing endpoints [24]
Secrets	“Rotate keys”	Key still present and was committed historically	Add mandatory git-history purge + prevent recurrence (pre-commit scanning) [25]
Stripe	“Use Checkout + webhooks”	No Stripe exists	Keep plan; add explicit raw body requirement and idempotency (webhook + create session) [26]

Single biggest architectural refinement to decide now

You should decide **how you will configure allowed origins for the postMessage bridge in production**. A safe default is:

- UI knows engine origin (e.g., `VITE_ENGINE_URL`)
- engine knows UI origin(s) via build-time env or server-served config
- both sides validate `event.origin` against a strict allowlist.

This implements MDN’s key guidance: strict `targetOrigin`, validate `origin` / `source`, validate message syntax. [6]

Go/no-go risk list and readiness conclusion

Top risks and mitigations

1. **Cross-window messaging trust boundary is open** (Critical): sending to `"*"` and missing origin checks. Mitigate with strict `targetOrigin` + receiver allowlist + schema validation. [6]
2. **Secrets exposure in git history** (Critical): rotate and purge; adopt secrets scanning and rotation runbook. [25]
3. **No server-side quote lock** (Critical): implement calculated/locked quotes in DB; payment only on locked quotes.
4. **No webhook verification / fulfillment logic** (Critical): verify signatures; use raw body; fulfill only via webhook; do not rely on success page. [27]
5. **No tenant isolation** (High): implement tenant-scoped queries and non-guessable IDs; never trust client tenant IDs. [18]
6. **Cookie-auth without CSRF strategy** (High): implement Signed Double-Submit or synchronizer token methods; enforce CSRF tokens on state-changing endpoints. [24]
7. **Upload endpoints not hardened** (High): allowlist types, size limits, authorized-only uploads, outside webroot storage, CSRF defenses. [12]

Readiness conclusion

- **Architecture match:** yes—the architecture you have is compatible with the manual/blueprint you're converging on.
- **Misinterpretations:** only minor (service gating), now corrected by your verification.
- **Amendments required before finalizing:** yes, mainly:
- tighten postMessage origin configuration (add allowlist; fix renderer receiver),
- add CSRF strategy for cookie-auth,
- elevate DB+auth into Phase 1 (not Phase 3) if operating as a real financial platform,
- formalize secrets purge + rotation governance.

If you adopt those amendments, the manual you pasted becomes a solid, realistic "start building now" playbook.

Top 10 immediate tasks

1. Remove the already-completed "service gate widening" item from the roadmap and add a regression test to ensure all 4 services remain supported.
2. Replace every `postMessage(..., "*")` with strict target origins in `renderer.tsx`, `viewer3d.tsx`, `catalog-item.tsx`, `PlannerFrame.tsx`, per MDN guidance. [28]
3. Add receiver-side `event.origin` (and ideally `event.source`) validation in `renderer.tsx` (your verification says it is missing). [5]
4. Decide and implement an origin allowlist strategy (build-time env/config), not arbitrary query-param trust, for the engine's expected UI origin. [29]
5. Rotate the RasterScan key and purge it from git history; add pre-commit scanning and a rotation runbook. [25]
6. Implement DB + migrations (minimum schema including `webhook_events` idempotency gate).

7. Implement auth with secure password hashing and cookie flags; document HTTPS requirements for `Secure` cookies. [30]
8. Add CSRF protections for cookie-authenticated, state-changing endpoints (projects save, quote lock, create session, upload). [24]
9. Implement pricing calculation + quote lock endpoints; replace `Estimate.tsx` rate=0 with server-calculated line items.
10. Implement Stripe Checkout Session creation + webhook verification + idempotency; fulfill only via webhook and treat success page as UI-only confirmation. [31]

---

[1] [2] [3] [4] [5] [6] [7] [9] [28] [29] Window: postMessage() method - Web APIs | MDN
https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage

[8] [25] Secrets Management - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

[10] [11] docs.stripe.com
https://docs.stripe.com/keys-best-practices

[12] [13] File Upload - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[14] [15] [24] Cross-Site Request Forgery Prevention - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

[16] [17] [30] Session Management - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[18] Multi Tenant Security - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html

[19] docs.stripe.com
https://docs.stripe.com/payments/checkout/custom-success-page

[20] [26] [27] docs.stripe.com
https://docs.stripe.com/webhooks

[21] [23] [31] Idempotent requests | Stripe API Reference
https://docs.stripe.com/api/idempotent_requests?utm_source=chatgpt.com

[22] How Checkout works
https://docs.stripe.com/payments/checkout/how-checkout-works?utm_source=chatgpt.com